

Technical Audit — GitHub Codebase RAG

Audit method: every claim below was checked against the actual source in this working tree (branch `feature/eval-suite-repo-scoping`, HEAD `2e1d9d9`), not the README. Two claims were additionally verified by executing code (see §7.1 and the embedding/LLM-backend cross-checks in §7.3–7.4). File paths are repo-relative; `file:line` refers to the line as of this audit.

1. Architecture Map & LangGraph State Machine

1.1 Node graph (`app/graph/rag_graph.py`)

Nine nodes, three conditional-edge routers, built with `langgraph.graph.StateGraph`:

```
intent --route_after_intent--> direct_answer --> END
      +--> retrieve --> rerank --> grade_chunks --> correct
      |
      | route_after_correct
      | +--> retrieve (loop, rewritten question)
      | +--> analyse --> answer --> self_critique
      | |
      | | route_after_critique
      | | +--> answer (loop, stricter_prompt)
      | | +--> END
```

Built in `build_graph()` (`rag_graph.py:337-393`):

Node	Function	Line	Responsibility
<code>intent</code>	<code>node_intent</code>	128	Classifies intent + runs the CRAG "retrieve gate"
<code>direct_answer</code>	<code>node_direct_answer</code>	141	Answers without retrieval (general/greeting questions)
<code>retrieve</code>	<code>node_retrieve</code>	165	Hybrid/vector retrieval, repo-scoped
<code>rerank</code>	<code>node_rerank</code>	180	Cross-encoder rerank to top-5
<code>grade_chunks</code>	<code>node_grade_chunks</code>	186	CRAG per-chunk relevance grading
<code>correct</code>	<code>node_correct</code>	208	CRAG decision: filter chunks or rewrite+retry
<code>analyse</code>	<code>node_analyse</code>	256	Builds file map / cross-refs / context brief

Node	Function	Line	Responsibility
answer	node_answer	262	Generates the final answer
self_critique	node_self_critique	273	Self-RAG grounding + utility check

1.2 Conditional edges — exact routing logic

route_after_intent ([rag_graph.py:314-318](#)):

```
def route_after_intent(state: RAGState) -> str:
    if not state.get("needs_retrieval", True):
        return "direct_answer"
    return "retrieve"
```

`needs_retrieval` is set upstream in `node_intent` by `check_need_retrieval()` (grader_agent — see §4.1). Note this is a **retrieve gate**, borrowed from Self-RAG's terminology, but it is wired in *before* the CRAG loop even runs — CRAG only ever sees a question that already passed the retrieve gate.

route_after_correct ([rag_graph.py:321-325](#)):

```
def route_after_correct(state: RAGState) -> str:
    if state.get("chunk_grades") is None and state["correction_attempts"] > 0:
        return "retrieve"
    return "analyse"
```

The signal for "we are retrying" is not a boolean flag but the *side effect* that `node_correct` nulled out `chunk_grades` (line 242) when it decided to rewrite the query. This is a slightly indirect state-machine idiom worth being able to explain: routing is inferred from the *absence* of a downstream field rather than an explicit `should_retry` flag.

route_after_critique ([rag_graph.py:328-331](#)):

```
def route_after_critique(state: RAGState) -> str:
    if state.get("needs_regeneration"):
        return "answer"
    return "end"
```

1.3 RAGState — every field ([rag_graph.py:69-88](#))

```
class RAGState(TypedDict):
    question : str
    original_question : Optional[str]
    session_id : str
    repo_name : Optional[str]
    chat_history : Optional[str]
    intent : Optional[str]
    intent_meta : Optional[dict]
    retrieved_chunks : Optional[list]
    reranked_chunks : Optional[list]
    analysis_brief : Optional[dict]
    final_answer : Optional[dict]
    correction_attempts : int
```

```

critique_attempts : int
needs_retrieval   : bool
chunk_grades      : Optional[list[str]]
stricter_prompt   : bool
needs_regeneration : bool
strict_mode       : bool

```

Field	Written by	Read by	Purpose
<code>question</code>	<code>query_repo</code> (init), <code>node_correct</code> (rewrite)	every retrieval/answer node	The <i>live</i> question — mutated during CRAG rewrite
<code>original_question</code>	<code>node_intent</code> (first write, <code>state.get(...)</code> or <code>state["question"]</code>), preserved thereafter	<code>node_grade_chunks</code> , <code>node_correct</code> , <code>node_self_critique</code>	Immutable copy of what the user actually typed, since <code>question</code> gets overwritten during rewrite
<code>session_id</code>	<code>query_repo</code> (init)	not read by any node directly (history is fetched <i>before</i> graph invocation)	Carried through state but the graph itself never queries Postgres — <code>query_repo</code> does that ahead of time (see §1.4)
<code>repo_name</code>	<code>query_repo</code> (init, from API/session binding)	<code>node_retrieve</code>	Drives the ChromaDB <i>where</i> filter and the per-repo BM25 index selection
<code>chat_history</code>	<code>query_repo</code> (init, pre-formatted string)	<code>node_direct_answer</code> , <code>node_answer</code>	Injected verbatim into prompts
<code>intent</code>	<code>node_intent</code>	<code>node_retrieve</code> , <code>node_answer</code> , <code>node_direct_answer</code>	Selects retrieval strategy + answer prompt instructions
<code>intent_meta</code>	<code>node_intent</code>	not consumed downstream in the graph (returned to caller only implicitly via <code>intent</code>)	Full classifier output dict (<code>intent</code> , <code>description</code> , <code>mode</code> , <code>question</code>) — effectively dead state past <code>node_intent</code>
<code>retrieved_chunks</code>	<code>node_retrieve</code> , cleared by <code>node_correct</code> on retry	<code>node_rerank</code>	Raw hybrid/vector search output before reranking
<code>reranked_chunks</code>	<code>node_rerank</code> , filtered by <code>node_correct</code> , cleared on retry	<code>node_grade_chunks</code> , <code>node_correct</code> , <code>node_analyse</code> , <code>node_self_critique</code>	Top-5 chunks post cross-encoder
<code>analysis_brief</code>	<code>node_analyse</code>	<code>node_answer</code>	Structured dict: <code>file_map</code> , <code>cross_refs</code> , <code>primary_files</code> , <code>context_summary</code>
<code>final_answer</code>	<code>node_direct_answer</code> , <code>node_answer</code>	<code>node_self_critique</code> , <code>query_repo</code> (return value)	The API-facing payload

Field	Written by	Read by	Purpose
<code>correction_attempts</code>	<code>node_correct</code>	<code>node_correct</code> (guards < 1), <code>route_after_correct</code>	Crag retry counter, capped at 1 retry
<code>critique_attempts</code>	<code>node_self_critique</code>	<code>node_self_critique</code> (guards < 1)	Self-RAG retry counter, capped at 1 retry
<code>needs_retrieval</code>	<code>node_intent</code>	<code>route_after_intent</code>	Retrieve-gate decision
<code>chunk_grades</code>	<code>node_grade_chunks</code> , nulled by <code>node_correct</code> on retry	<code>route_after_correct</code> (as a sentinel, see §1.2)	Per-chunk <code>relevant/ambiguous/irrelevant</code> labels
<code>stricter_prompt</code>	<code>node_self_critique</code>	<code>node_answer</code>	Switches the answer prompt to a stricter, context-only mode on regeneration
<code>needs_regeneration</code>	<code>node_self_critique</code>	<code>route_after_critique</code>	Self-RAG regenerate decision
<code>strict_mode</code>	<code>query_repo</code> (init, from API request)	<code>node_grade_chunks</code> , <code>node_self_critique</code>	Global kill-switch for Crag grading + Self-RAG critique (see §4.3)

1.4 Postgres session memory (`app/memory/db.py`, `models.py`, `session.py`)

Schema (`models.py`): `chat_sessions(id UUID PK, repo_url, created_at)` 1—N `chat_messages(id UUID PK, session_id FK, role, content, intent, sources JSON-as-text, created_at)`, cascade-delete on the relationship (`models.py:21-25`).

Repo binding. A session is bound to a repo by storing the raw GitHub URL on `ChatSession.repo_url` — set at creation time in `/ingest` (`create_session(repo_url=req.url)`, `routes.py:108`) or lazily via `bind_session_repo()` (`session.py:114-129`). The **repo name** used for scoping is never stored — it's derived on every read via `extract_repo_name()` (`session.py:86-89`):

```
def extract_repo_name(repo_url: str | None) -> str | None:
    if not repo_url:
        return None
    return repo_url.rstrip("/").split("/")[-1].removesuffix(".git")
```

This is the exact same derivation `ingest_repo()` uses to name the BM25 pickle (`rag_graph.py:419`), so the two stay consistent by construction — but it means the "repo identity" is a filename-derived string, not a stable ID; two different GitHub orgs' repos named `utils` would collide.

Where scoping is actually enforced: - `/chat` resolves `repo_name` from the session (`get_session_info` → `extract_repo_name`) unless the request explicitly overrides it (`routes.py:142-145`). - `node_retrieve` turns that into a ChromaDB metadata filter: `filter_meta = {"repo": repo_name} if repo_name else None` (`rag_graph.py:166-167`), passed straight into `collection.query(..., where=filter_meta)` in `vector_search()` (`vector_store.py:121-126`). With `repo_name=None` the filter is `None` — **not** an empty dict — so an unscoped session searches the *entire* collection across every ingested repo. - BM25 scoping is a separate, non-Chroma mechanism: `_get_bm25_retriever(repo_name)`

([rag_graph.py:97-121](#)) looks up `repos/bm25_{repo_name}.pkl` on disk, not a metadata filter over a shared index. Each repo effectively gets its own standalone BM25Okapi instance built from only that repo's chunks — so BM25 scoping is structural (separate corpora) while vector scoping is filter-based (shared corpus, `where` clause).

Server restart mid-conversation. Chat history is durable (Postgres), so a restart loses nothing there — `get_history(session_id)` ([session.py:45-62](#)) just re-queries on the next turn. What is lost is the in-process caches: - `_bm25_retrievers: dict[str, BM25Retriever]` and `_fallback_bm25_retriever` ([rag_graph.py:94-95](#)) reset to empty. The next query for a given repo pays a one-time `BM25Retriever.load()` deserialization cost from the pickle on disk — functionally transparent (the pickle persists the corpus + fitted `BM25Okapi`), but this is exactly the mechanism that breaks under multiple worker processes (see §7). - `_compiled_graph` ([rag_graph.py:396](#)) and the reranker's `_cross_encoder` singleton ([reranker.py:39](#)) are rebuilt lazily on first use post-restart — the LangGraph compile step and the `CrossEncoder(...)` model load both re-run, adding latency to the first post-restart query only. - ChromaDB is a `PersistentClient(path=CHROMA_PATH)` ([vector_store.py:31](#)) — data is on disk, unaffected by restart.

2. Ingestion & Multi-Tier AST Chunking Pipeline

2.1 `clone_repo.py`

`clone_repo(url)` ([clone_repo.py:16-41](#)) shallow-clones (`depth=1`) into `REPOS_PATH/{repo_name}` via GitPython, and **skips cloning entirely if the target directory already exists** ([clone_repo.py:26-28](#)) — there is no re-clone/pull-to-update path, so re-ingesting the same repo name after the source has changed silently reuses stale files on disk (it will still re-embed whatever's there, but "whatever's there" is frozen at first-clone time).

2.2 `scan_repo.py`

`scan_repo(repo_path)` ([scan_repo.py:71-128](#)) walks the tree with `os.walk`, pruning `EXCLUDED_DIRS` **in-place** on the `dirs` list (`dirs[:] = [...]`, [scan_repo.py:96](#)) so `node_modules/.git/target/etc.` are never descended into (not just filtered after the fact — this matters for clone size). Filters: 23 `SUPPORTED_EXTENSIONS` ([scan_repo.py:10-35](#)), an `EXCLUDED_FILES` lockfile denylist ([scan_repo.py:54-64](#)), and a hard `MAX_FILE_SIZE_KB = 200` ([scan_repo.py:66](#)) skip for large generated files.

2.3 `chunker.py` — the three tiers

Tier 1 — tree-sitter AST. Each language maps to an *individual* pip package (`_TS_LANG_MAP`, [chunker.py:28-52](#)), e.g. `"py": "tree_sitter_python"`, dynamically `__import__`-ed in `_load_ts_language()` ([chunker.py:272-304](#)) and cached in `_lang_cache`. TypeScript/TSX are a special case — one package (`tree_sitter_typescript`) exposes two grammar functions, `language_typescript()` vs `language_tsx()` ([chunker.py:289-292](#)).

Parsing is byte-oriented, not string-oriented, which matters correctness-wise for any file with multi-byte UTF-8 characters (emoji, non-Latin identifiers/comments):

```
code_bytes = code.encode("utf-8") # chunker.py:313
tree = parser.parse(code_bytes)
...
chunk_bytes = code_bytes[node.start_byte:node.end_byte] # chunker.py:328
text = chunk_bytes.decode("utf-8", errors="ignore")
```

tree-sitter's `start_byte/end_byte` are byte offsets; slicing the *decoded string* with them (a bug the commit history shows was fixed — see [9da3685/9eb7f06](#)) would misalign on any non-ASCII content. Slicing `code_bytes` first, then decoding the resulting sub-slice, is the correct approach.

Only *top-level* AST nodes are chunked (`for node in tree.root_node.children`, [chunker.py:324](#)) and only if `node.type` is in `_AST_NODE_TYPES` ([chunker.py:54-80](#)) — a fixed set like `function_definition`, `class_definition`, `impl_item`, `rule_set` (CSS), `pair` (JSON/YAML), `section` (Markdown). Consequence: a Python method nested inside a class is **not** its own chunk — the entire class is one chunk (since `function_definition` only matches top-level, and a method's parent is `class_definition`, not `root_node`). Large classes therefore become large, undifferentiated chunks. Chunk naming (`_get_node_name`, [chunker.py:341-364](#)) looks for a child `identifier/tag_name/property_name/key` node, with hand-written special cases for JSON/YAML `pair`, CSS `rule_set`, and Markdown `section`; anything else names itself `"unknown"`.

Tier 2 — regex. Per-extension pattern lists (`REGEX_PATTERNS`, [chunker.py:84-198](#), 17 languages) are OR-combined into one compiled regex and matched line-by-line; every matching line becomes a new chunk boundary ([chunker.py:371-396](#)). This only runs when Tier 1 either isn't available for that extension or returned zero chunks (e.g., no top-level constructs matched).

Tier 3 — sliding window, the universal fallback ([chunker.py:408-424](#)):

```
CHUNK_LINES = 100
CHUNK_OVERLAP = 20
...
start += CHUNK_LINES - CHUNK_OVERLAP # advances 80 lines per chunk, 20-line overlap
```

Guarantees every file produces *some* chunk regardless of language/syntax, at the cost of chunk boundaries that ignore code structure entirely.

`chunk_file()` ([chunker.py:211-243](#)) tries tiers in strict fallback order and every chunk carries a uniform metadata dict via `_make_chunk()` ([chunker.py:431-448](#)): `repo`, `file_path`, `language`, `chunk_type`, `chunk_name`, `start_line`, `end_line`, `size_kb`, `chunking_tier`.

2.4 Vector store & BM25

vector_store.py — Chroma `PersistentClient` at `CHROMA_PATH` (default `./chroma_db`), single collection `"codebase"` with `hnsw:space: cosine` ([vector_store.py:35-41](#)). `index_chunks()` ([vector_store.py:47-85](#)) IDs each chunk by `hashlib.md5(text.encode()).hexdigest()` — this is **content-addressed deduplication**: identical code appearing in two files (or re-ingested twice) collapses to one Chroma row via `upsert()`, and a `seen_ids` set additionally dedupes within a single ingestion batch ([vector_store.py:63,66-69](#)). `_sanitize_metadata()` ([vector_store.py:88-99](#)) coerces any non-`str/int/float/bool` metadata value to `str`, since Chroma rejects other types.

Embeddings are **not** `nomic-embed-text` in the code path that actually runs — see §7.3 for the full discrepancy. `embedder.py` hardcodes `langchain_google_genai.GoogleGenerativeAIEmbeddings`, with `EMBED_MODEL` read from env (`text-embedding-004` default in code, `gemini-embedding-001` in this repo's own `.env`). `embed_documents()` batches at 32 texts/call ([embedder.py:50-56](#)) to avoid API payload/timeout limits.

bm25.py — `BM25Retriever` wraps `rank_bm25.BM25Okapi.tokenize()` ([bm25.py:16-23](#)) lowercases and splits on whitespace **and** code punctuation (`()[]{}<>.,;:=+*/\\"'`), dropping single-character tokens — a code-aware tokenizer, not a prose tokenizer (`soself.foo()` → `["self", "foo"]`, not one token). Persistence is a flat pickle of `{"chunks": [...], "bm25": BM25Okapi_instance}` ([bm25.py:88-104](#)) (`app/retrieval/bm25.py:88`) — the entire corpus is duplicated into every repo's `.pkl` file (no shared storage), which is fine at

hobby-project scale but means disk usage grows with $O(\text{repos} \times \text{avg_repo_chunks})$ and every repo's full text sits in-process once loaded.

3. Hybrid Retrieval & Reranking Pipeline

3.1 Intent classification (intent_agent.py)

Two modes gated by `INTENT_MODE` env var (default `"rules"`, [intent_agent.py:26](#)): - **Rules** (`_classify_rules`, [intent_agent.py:80-98](#)): keyword-count voting across 5 intents (`code_search`, `explain`, `trace_flow`, `architecture`, `debug`), `max()` picks the highest-scoring intent, ties broken by dict iteration order (first-defined wins), zero matches → `DEFAULT_INTENT = "explain"`. - **LLM** (`_classify_llm`, [intent_agent.py:124-148](#)): `ChatGoogleGenerativeAI` with `temperature=0`, single-label prompt, falls back to `_classify_rules()` on any exception or unrecognized response.

Correction to the framing in the audit brief: intent does **not** dynamically tune `n_results`. `RETRIEVAL_N` is a single module constant (30, [retrieval_agent.py:26](#)) applied identically across all five strategies. What intent actually tunes is the **RRF weight split** between vector and BM25 (see table), plus two structural variants:

Intent	vector_weight	bm25_weight	Notes
code_search	0.4	0.6	Favors exact identifier match (retrieval_agent.py:82-98)
explain	0.6	0.4	Favors semantic similarity (retrieval_agent.py:101-117)
trace_flow	0.5	0.5	Plus <code>_cap_per_file(max_per_file=2)</code> post-filter for cross-file diversity (retrieval_agent.py:120-139)
architecture	—	—	Vector-only , calls <code>vector_search()</code> directly, bypasses BM25/hybrid entirely (retrieval_agent.py:142-155)
debug	0.3	0.7	Heaviest BM25 bias, for error strings/identifiers (retrieval_agent.py:158-175)

3.2 Hybrid RRF (hybrid.py)

score(d) = $\sum \text{weight_system}(d) \times 1 / (\text{RRF_K} + \text{rank}(d) + 1)$

Implemented at [hybrid.py:56-74](#): both vector and BM25 result lists are enumerated (0-indexed `rank`), each hit contributes `weight × 1/(RRF_K + rank + 1)` to a shared score map keyed by chunk identity, and a chunk found by both systems accumulates *both* contributions. `RRF_K = 60` ([hybrid.py:20](#)) is the standard damping constant from the original RRF

paper — it flattens the influence of rank position (a large `k` means rank 1 vs rank 5 barely differ in score; a small `k` would make rank 1 dominate). Note this implementation multiplies each system's $1/(k+rank+1)$ term by a *weight* before summing — that's a weighted variant of RRF, not the textbook unweighted formula (which sums the two $1/(k+rank+1)$ terms unweighted); worth being explicit about this distinction if asked "is this the RRF formula from the paper."

Dedup key: `f"{file_path}::{start_line}"` ([hybrid.py:52-54](#)) — identifies a chunk by *where it starts in a file*, not by content hash. This is deliberately different from the vector store's MD5-content-based ID; it lets vector and BM25 hits (which may have re-derived the "same" chunk independently but as distinct dict objects) merge as one row for ranking purposes. Caveat: the key has no `repo` component, so with `filter_meta=None` (unscoped, cross-repo query) two different repos' `app/main.py:1` chunks would collide under one RRF key and silently merge scores/sources — see §7.

`simple_merge()` ([hybrid.py:95-119](#)) is a documented, unused-by-the-graph alternative — vector-first concatenation with the same dedup key, no RRF math. Confirmed via `grep` that `node_retrieve`'s call chain never reaches it; it exists as an escape hatch / comparison baseline.

3.3 Reranker (`reranker.py`)

Two backends selected by `RERANKER_BACKEND` env (default `"cross_encoder"`, [reranker.py:31](#)): - **CrossEncoder** (`sentence_transformers.CrossEncoder`, model `BAAI/bge-reranker-base` by default): scores (`question`, `chunk_text`) pairs jointly through one transformer forward pass per pair ([reranker.py:58-80](#)) — far more accurate than the bi-encoder cosine similarity used for the initial vector search, because the model attends across the query and document jointly instead of comparing two independently-computed embeddings. - **Ollama LLM reranker** (`langchain_ollama.OllamaLLM`, [reranker.py:87-133](#)): prompts the LLM to emit a 0–10 relevance score per chunk, one LLM call per chunk — used only as a fallback when the CrossEncoder can't load.

`rerank()` ([reranker.py:143-188](#)) tries CrossEncoder → Ollama → identity-slice, in that order, and **skips reranking entirely if `len(chunks) <= top_k`** ([reranker.py:164-166](#)) since there's nothing to discriminate.

Why retrieve 25–30 to keep 5: hybrid search (RRF over two independently-imperfect rankers) is optimized for *recall* — cheaply casting a wide net that's likely to contain the right chunk somewhere in the top 20–30. The cross-encoder is expensive per-pair (a full transformer pass, not a cached embedding lookup) but far more *precise* at judging true relevance — so the architecture spends the expensive model only on a small candidate set that recall has already narrowed down, rather than running it over the entire corpus. This is the standard two-stage "retrieve-then-rerank" IR pattern.

4. Self-Correction Layer (Crag + Self-RAG)

4.1 `grader_agent.py`

`check_need_retrieval()` ([grader_agent.py:77-104](#)) — the retrieve gate. Two layers: 1. A hardcoded keyword override list, `_CODEBASE_KEYWORDS` ([grader_agent.py:33-43](#)) — e.g. `"stategraph"`, `"bm25"`, `"chromadb"`, `"routes.py"` — any hit **forces True** without an LLM call at all. This exists so meta-questions about *this specific RAG system's own internals* (which an LLM gate might otherwise misjudge as "general knowledge") are never wrongly routed to `direct_answer`. 2. Otherwise, an LLM yes/no gate (`_RETRIEVE_GATE_PROMPT`, [grader_agent.py:45-62](#)) using word-boundary regex parsing of the response (`re.findall(r'\b\w+\b', response)`) rather than exact string match — defensive against the LLM adding punctuation/preamble. **Fails open:** any exception defaults to `True` ([grader_agent.py:103-104](#)) — retrieval is the "safe" failure mode (worst case, wasted retrieval; never a silently-wrong direct answer to a codebase question).

`grade_chunk_relevance()` ([grader_agent.py:106-132](#)) — per-chunk three-way classification (`relevant/ambiguous/irrelevant`) via `_CHUNK_GRADER_PROMPT` ([grader_agent.py:64-75](#)), same word-boundary parsing. Unlike the retrieve gate, this **fails toward inclusion**: an unparseable response defaults to `"ambiguous"` (kept), and an exception defaults to `"relevant"` ([grader_agent.py:131-132](#)) — i.e., grading failures never discard a chunk.

`node_correct()` ([rag_graph.py:208-252](#)) consumes the grades: chunks graded `relevant` or `ambiguous` survive ([rag_graph.py:213-216](#)); only `irrelevant` is dropped. If `zero` chunks survive and `correction_attempts < 1`, it asks the LLM to rewrite the query for better keyword/API-name coverage ([rag_graph.py:226-232](#)), sets `question` to the rewritten text, increments the counter, and nulls `retrieved_chunks/reranked_chunks/chunk_grades` to force a clean re-run of the retrieve→rerank→grade sequence. Otherwise it restores `question` to `original_question` (undoing any rewrite) and proceeds with whatever survived filtering — including `zero` chunks, if the retry also failed; there is no third attempt, and `node_analyse/node_answer` degrade gracefully to their empty-chunks branches (`_empty_brief`, [analysis_agent.py:198-207](#); the "I could not find relevant code..." canned response, [answer_agent.py:137-144](#)).

4.2 critique_agent.py

`check_grounding()` ([critique_agent.py:61-99](#)) — builds a context string from every chunk's `file_path` + text, asks the LLM a yes/no "is every claim in this answer supported by this context" question (`_GROUNDING_PROMPT`, [critique_agent.py:32-46](#)). Short-circuits to `True` if there are no chunks at all (line 67-70) — an empty-context answer can't be judged for grounding here, so that responsibility is deferred to the utility check. Fails open (`True`) on exception ([critique_agent.py:97-99](#)).

`check_utility()` ([critique_agent.py:101-126](#)) — separate yes/no LLM call, "does this answer actually address the question" (`_UTILITY_PROMPT`, [critique_agent.py:48-59](#)). Also fails open.

`node_self_critique()` ([rag_graph.py:272-308](#)) runs both checks against `original_question` (not the possibly-rewritten `question`), and if **either** fails and `critique_attempts < 1`: sets `stricter_prompt=True`, `needs_regeneration=True`, increments the counter. `route_after_critique` then sends the graph back to `node_answer` — **not** back to retrieval — so a second attempt re-generates from the *same* `reranked_chunks` with a stricter system prompt appended ([answer_agent.py:87-88](#)) that forbids extrapolation beyond the shown code. This means Self-RAG here only fixes hallucination caused by the LLM's own liberties with correct context — it cannot fix an answer that's ungrounded because the *retrieved* context was insufficient (that failure mode is CRAG's job, and CRAG already ran upstream, so by the time Self-RAG fires, the correction budget for retrieval is typically spent).

4.3 strict_mode

A single boolean threaded from the `/chat` and `/query` request bodies down to `RAGState`. It short-circuits exactly two nodes: - `node_grade_chunks` ([rag_graph.py:189-193](#)): skips all per-chunk LLM grading, stamps every chunk `"relevant"` — meaning `node_correct` will never see zero survivors and the CRAG rewrite loop is structurally unreachable in fast mode. - `node_self_critique` ([rag_graph.py:274-280](#)): skips both grounding/utility checks, sets `needs_regeneration=False` unconditionally — the Self-RAG loop is structurally unreachable too.

The graph's node/edge *topology* is identical in both modes (the same nodes execute in the same order); what changes is only whether those two nodes perform real LLM-call work or become no-ops. This means fast mode still pays the reranker's CPU cost and the intent/analysis/answer LLM calls — the savings are specifically 2 grading LLM calls per chunk (up to 5) + up to 2 critique LLM calls, i.e., strict mode's latency overhead scales with chunk count while fast mode's does not.

5. API & System Integration

5.1 `routes.py`

FastAPI app with a `lifespan` context manager that calls `init_db()` on startup ([routes.py:26-29](#)) — `Base.metadata.create_all()` under the hood ([db.py:32-37](#)), i.e., **no Alembic migrations actually run despite Alembic being a listed dependency** (see §7). CORS is wide open (`allow_origins=["*"]`, [routes.py:39-44](#)) — fine for a local Streamlit-to-FastAPI setup, not something to ship as-is behind a public origin.

- **POST `/ingest`** ([routes.py:101-112](#)) → `ingest_repo(url)` (clone → scan → chunk → embed/index → BM25 build, all synchronous/CPU-and-network-bound work called directly inside an `async def` route with no `run_in_executor`) → creates a new Postgres session bound to that URL via `create_session()`.
- **POST `/query`** ([routes.py:115-133](#)) — explicitly stateless: passes a fixed all-zero UUID as `session_id` into `query_repo()`. Since nothing ever calls `save_message()` for that UUID, `get_history()` against it always returns empty — safe, but the "shared sentinel session ID" pattern is a small landmine if a future change starts persisting `/query` turns.
- **POST `/chat`** ([routes.py:136-170](#)) — resolves `repo_name` from the session if not explicitly given, creates a session if none provided, saves the user turn *before* calling `query_repo()` and the assistant turn after, in two separate commits (not one transaction) — the trade-off being the user's message is durably recorded even if generation subsequently throws (caught as a 500 or a 400 for a missing-BM25 `RuntimeError`).
- **POST `/reset`** ([routes.py:183-191](#)) → `reset_all()` from `reset.py`: truncates `chat_messages/chat_sessions`, deletes and recreates the Chroma collection *and* removes the entire `chroma_db/` directory ([reset.py:30-44](#)), and wipes everything under `repos/` (clones + BM25 pickles) ([reset.py:47-59](#)). This is an irreversible, full-system wipe with no confirmation step or scoping (no "reset just this repo") — appropriate for a dev/demo tool, not for anything multi-tenant.

5.2 `frontend/streamlit_app.py`

Plain `requests`-based client against `API_URL = "http://127.0.0.1:8080"` (hardcoded, not env-driven). State lives in `st.session_state: session_id, active_repo, messages`. The sidebar's "Past sessions" list re-fetches `/sessions` on every rerun and lets the user click into any session, which pulls `/sessions/{id}/history` and replaces `st.session_state.messages` — this is how continuity survives a browser refresh (Streamlit reruns the whole script top-to-bottom on every interaction, so there's no client-side session persistence beyond what Postgres backs). `active_repo` is only ever set from `data.get("repo")` after a fresh `/ingest` or from `s.get("repo_name")` when picking a past session — the frontend never lets a user manually retarget a session to a different repo except by starting a new one.

6. Evaluation Suite & Benchmarking

Three levels, matching the description in the audit brief closely (this part of the codebase is unusually well-organized relative to the rest):

Level 1 — Component (`component_eval.py`)

- `eval_intent_classifier()` — rule-based vs LLM-based accuracy against `GOLDEN_DATASET`'s labeled `intent` field, via `accuracy_score()` exact-match ([component eval.py:27-43](#)).
- `eval_isolated_retrieval()` — vector-only vs BM25-only, each queried **directly** (bypassing `hybrid_search/retrieve`) to isolate each system's raw file-level P@k/R@k/MRR ([component eval.py:45-109](#)).
- `eval_reranker_impact()` — same query set through the full `retrieve()` (hybrid, intent-aware) before vs after `rerank()`, delta on P@1 and MRR ([component eval.py:111-144](#)) — this is the number that answers "does the cross-encoder actually help."
- `eval_chunk_grader()` — `grade_chunk_relevance()` accuracy against a small 4-item hand-labeled `GRADER_TEST_SET` ([golden dataset.py:137-158](#)).

Level 2 — Pipeline / RAG Triad ([pipeline_eval.py](#))

- `eval_hybrid_pipeline()` — full `retrieve()+rerank()` P@k/R@k/MRR (the "real" retrieval number, as opposed to Level 1's isolated ablations).
- `eval_retrieve_gate()` — `check_need_retrieval()` accuracy against `RETRIEVE_GATE_TEST_SET` (4 hand-labeled yes/no cases, [golden dataset.py:161-178](#)).
- `eval_self_rag_critique()` — a hand-built grounded vs ungrounded answer pair, checks `check_grounding()` correctly accepts one and rejects the other ([pipeline eval.py:78-95](#)) — a 2-example smoke test, not a statistically meaningful accuracy number (scored `1.0` or `0.5`, never lower).
- `eval_rag_triad_deepeval()` — runs full `graph.invoke()` per golden question, then scores the real chunks/answer with DeepEval's `FaithfulnessMetric`, `AnswerRelevancyMetric`, `ContextualRelevancyMetric`, all backed by a custom `GoogleGeminiJudge` (§6.1).

Level 3 — Application E2E ([app_eval.py](#))

- `eval_end_to_end()` — full graph invocation per golden question (`strict_mode=True`), computes P@k/R@k/MRR on retrieved files plus RAGAS-style `context_precision/context_recall` ([metrics.py:65-96](#)) and reuses `check_grounding/check_utility` as binary "faithfulness"/"utility" generation metrics ([metrics.py:48-63](#)) — **this reuses the exact same LLM-graders that gate the pipeline itself as the evaluation's ground truth**, which is a meaningful limitation (see §7).
- `eval_strict_mode_comparison()` — runs the same 2 golden questions through the graph twice, once per `strict_mode` value, measuring wall-clock latency to produce the "fast mode speedup" percentage.
- `eval_geval_code_correctness()` — DeepEval's `GEval` (LLM-graded free-form rubric: "does the response use correct function/class names and cite files accurately") on one sample question.

6.1 `deepeval_judge.py` and `golden_dataset.py`

`GoogleGeminiJudge(DeepEvalBaseLLM)` ([deepeval judge.py:16-76](#)) lets every DeepEval metric run against Gemini instead of requiring an OpenAI key — the interesting engineering detail is the retry backoff wrapping every `generate/a_generate` call: on a 429/quota error it sleeps `(attempt+1)*7` seconds (7s, 14s, 21s...) up to 5 attempts, tuned to Gemini's free-tier ~10 RPM limit ([deepeval judge.py:47-49](#)). Correctly extracts `str(response.content).strip()` from the `AIMessage` — see §7.1 for why this matters by contrast.

`GOLDEN_DATASET` ([golden dataset.py:8-134](#)) is 24 **hand-authored** questions about this repo's own source code, grouped by the same 5 intents the classifier predicts, each with `question`, `intent`, `expected_files` — i.e. it's a self-referential eval set (the RAG system evaluated on questions about itself), which is why `_CODEBASE_KEYWORDS` in the grader exists (§4.1) — the eval questions are exactly the kind of meta-question that keyword list is designed to force-retrieve for. Two smaller hand-labeled sets, `GRADER_TEST_SET` and `RETRIEVE_GATE_TEST_SET`, support the Level 1/2 component checks.

`create_llm_test_case()` ([golden_dataset.py:180-196](#)) wraps a question/answer/context triple into a DeepEval `LLMTestCase`, swallowing any construction error into a silent `None` return (callers check `if test_case:` before using it).

7. Gaps and Weak Points

This section is deliberately unsparing — these are the things worth pre-empting in an interview, in rough order of severity. Findings marked **[verified by execution]** were not just read but actually reproduced against the installed dependency versions in this environment (`langchain-core` 1.5.5, `langchain-google-genai` 4.3.4).

7.1 Confirmed crash: `AIMessage.strip()` in the core answer path [verified by execution]

`ChatGoogleGenerativeAI.invoke()` (like every LangChain chat model) returns an `AIMessage`, not a string — `BaseChatModel.invoke` is typed `-> AIMessage` in `langchain_core`. `AIMessage` has no `.strip()` method. Three call sites treat the return value as a string anyway:

- `app/agents/answer_agent.py:153,170` — `response = llm.invoke(prompt)` then, *outside* the surrounding `try/except` (which only wraps the `.invoke()` call itself, lines 151-162), `"answer": response.strip()` at line 170. I reproduced this directly: `RAISED: AttributeError: 'AIMessage' object has no attribute 'strip'` by monkeypatching `answer_agent._get_llm` to return a stub whose `.invoke()` returns a real `AIMessage`, then calling `answer()` with one non-empty chunk. Since the `.strip()` call is outside the `try/except`, this is an **uncaught `AttributeError`** that propagates out of `node_answer`, crashes the LangGraph invocation, and surfaces as a generic 500 from `/chat` or `/query` (caught only by FastAPI's outer `except Exception` in `routes.py`).
- `app/graph/rag_graph.py:153` (`node_direct_answer`) — `response = llm.invoke(prompt).strip()`. This is the path for every "no retrieval needed" question (general programming questions, greetings) — i.e. a large, common fraction of real traffic would hit this if exercised end-to-end.
- `app/graph/rag_graph.py:233` (`node_correct`, the CRAG query-rewrite branch) — same pattern.

By contrast, `grader_agent.py`, `critique_agent.py`, `intent_agent.py`, and `deepeval_judge.py` all correctly do `res.content if hasattr(res, "content") else str(res)` before calling string methods — the commit history (9da3685 fix: update Gemini LLM integration ... grader response parsing) shows this exact class of bug was found and fixed in the grader/critique agents but **the fix was never applied to `answer_agent.py` or the two call sites in `rag_graph.py`**. This is worth stating plainly and owning in an interview rather than being caught off guard by it — it's a real, currently-present defect, not a hypothetical.

Downstream consequence: `pipeline_eval.eval_rag_triad_deepeval()` and `app_eval.eval_strict_mode_comparison()/eval_geval_code_correctness()` call `graph.invoke()` **without** a `try/except` (unlike `app_eval.eval_end_to_end()`, which does wrap it and would silently record zeros for the affected question instead of crashing the whole run).

7.2 Silent fail-open/fail-closed exception handling, inconsistently applied

Every LLM-backed grading function catches broad `Exception` and returns a hardcoded default (§4.1, §4.2) — a defensible pattern (never let a transient API error crash the pipeline), but the defaults aren't uniformly biased the same direction, and none of them are logged anywhere more durable than `loguru` — there's no metric/counter distinguishing "the grader said irrelevant" from "the grader crashed and we assumed relevant." In production this makes it impossible to tell, from the outside, whether a low CRAG-rewrite rate means retrieval is genuinely good or means the grader is silently failing every call.

7.3 README documents a local-inference path that the code doesn't implement

The README's "Local Inference" bullet and `.env.example`-style block advertise `OLLAMA_BASE_URL` + `EMBED_MODEL=nomic-embed-text` for fully local, private embeddings. `app/embeddings/embedder.py` has **no Ollama/local embedding backend at all** — it unconditionally builds `GoogleGenerativeAIEmbeddings(model=EMBED_MODEL, ...)` ([embedder.py:32-35](#)). Setting `EMBED_MODEL=nomic-embed-text` as the README instructs would pass that string to Google's embeddings API, which doesn't recognize it, and would fail at the first `embed_documents()/embed_query()` call. This repo's actual `.env` runs `EMBED_MODEL="gemini-embedding-001"` — cloud embeddings, not local, contradicting the "completely private" framing.

7.4 "Ollama LLM" is only wired into the reranker, nowhere else

Similarly, the README claims local inference via Ollama's `qwen2.5:3b` for generation. Grepping every agent confirms `answer_agent.py`, `grader_agent.py`, `critique_agent.py`, and `intent_agent.py`'s LLM mode **all hardcode ChatGoogleGenerativeAI** — there is no `if LLM_BACKEND == "ollama"` branch anywhere for the generation/grading/critique/intent LLM calls. The only place `langchain_ollama` is actually imported and used is `reranker.py`'s fallback backend (`_rerank_ollama`, only invoked if the CrossEncoder fails to load). Be ready to say precisely which one component supports Ollama today (the reranker) versus which the README implies (everything).

7.5 Alembic is a dependency and a "codebase keyword" but isn't actually used

`alembic>=1.18.4` is pinned in `requirements.txt`, and `"migration"/"alembic"` are literally in `grader_agent._CODEBASE_KEYWORDS` ([grader_agent.py:36](#)) as terms that should force-retrieve codebase context — but there is no `alembic/` directory, no `alembic.ini`, and `init_db()` uses `Base.metadata.create_all()` directly ([db.py:32-37](#)). Schema changes today have no migration path — adding a column to `ChatMessage` in production would require a manual `ALTER TABLE` or a destructive drop/recreate; there's no versioned upgrade/downgrade story despite the dependency being present.

7.6 In-memory BM25 cache doesn't survive or share across multiple worker processes

`_bm25_retrievers/_fallback_bm25_retriever` ([rag_graph.py:94-95](#)) are plain module-level globals. Under `uvicorn --workers N` or any multi-process deployment, each worker holds an independent copy, each lazily reloading the same pickle from disk on first use — functionally correct (the source of truth is the file) but memory cost multiplies by worker count, and there's a window right after a fresh `/ingest` where only the worker that handled the ingest request has the freshly-built retriever in memory (`ingest_repo()` updates the global cache directly, [rag_graph.py:435-437](#)) — other workers keep serving the *old* pickle from disk until their own cache misses and reloads, or forever if the repo name was already cached pre-ingest. `main.py`'s `uvicorn.run(..., reload=True)` (dev mode, single process) sidesteps this entirely, so it wouldn't surface in the way this project is actually run today — but it's a real latent issue for anyone scaling this past a single process.

7.7 Synchronous, CPU-bound work inside `async def` routes

`node_retrieve` (BM25 scoring), `node_rerank` (CrossEncoder forward passes), and every embedding call are synchronous, CPU/GPU-bound Python calls, invoked from `query_repo()` — an `async def` — via a plain, blocking `graph.invoke(initial_state)` ([rag_graph.py:486](#)) with no `run_in_executor`/thread-pool offload. Under FastAPI's single-threaded asyncio event loop, a single slow rerank call blocks *all* other concurrent requests (including unrelated `/health` checks) for its duration. This is invisible at demo scale (one user, one request at a time) and becomes the first concurrency bottleneck under any real load.

7.8 Fixed retry caps and a rewrite loop that can't fully recover

Both CRAG (`correction_attempts < 1`) and Self-RAG (`critique_attempts < 1`) allow exactly **one** retry, hardcoded, not configurable via env. And structurally, Self-RAG's regeneration only ever calls `node_answer` again with the same `reranked_chunks` — it never loops back to retrieval — so if an ungrounded answer results from genuinely insufficient context (as opposed to the LLM inventing details it wasn't given), the one stricter-prompt retry can't source better context; it can only ask the model to hedge more honestly (or produce a shorter, more conservative answer from the same limited material).

7.9 RRF dedup key collision risk across unscoped, multi-repo queries

`hybrid_search`'s chunk identity key is `f"{file_path}::{start_line}"` with no `repo` component ([hybrid.py:52-54](#)). When a query is unscoped (`repo_name=None`, `filter_meta=None`), vector search legitimately spans every ingested repo's chunks in one collection. Two different repos that happen to share a relative path and starting line (e.g., both have a `README.md` chunk starting at line 1, or both scaffolded from the same boilerplate) would collide under this key, and the second repo's chunk to be processed would silently overwrite the `chunk_map` entry's `text/metadata` while their RRF scores merge — a wrong-content, wrong-score result that would be hard to notice without dedicated testing.

7.10 `RERANKER_TOP_K` env var is dead for the main pipeline

`reranker.py`'s public `rerank()` defaults `top_k` to `TOP_K_DEFAULT`, itself read from `RERANKER_TOP_K` env (set to 5 in this repo's `.env`) — but `node_rerank` in the graph calls `rerank(state["question"], state["retrieved_chunks"] or [], top_k=5)` with an **explicit, hardcoded 5** ([rag_graph.py:181](#)), so changing `RERANKER_TOP_K` in `.env` has zero effect on the actual LangGraph pipeline — it would only matter for standalone calls to `rerank()` that omit `top_k` (e.g. ad hoc scripts).

If I were reviewing this as a senior engineer, the first three things I'd ask the author to defend

1. **"Walk me through what happens when a user asks a general question with no codebase context."** This routes to `node_direct_answer`, which — as currently written — throws an uncaught `AttributeError` on the `.strip()` call (§7.1) rather than returning an answer. I'd want to see this either already fixed or the author immediately recognizing it live and explaining the one-line fix (`llm.invoke(prompt).content.strip()` or route through a `StrOutputParser`).
2. **"Your README advertises a fully local/private deployment option. Is that true today?"** — a good test of whether the author can distinguish what's implemented from what's aspirational/documented-but-not-wired (§7.3/7.4). This is a completely normal state for an evolving side project; the question is whether the author *knows* the gap exists.
3. **"How would this behave with two workers under load, and what's your plan for the BM25 in-memory cache and the CPU-bound rerank call blocking the event loop?"** (§7.6/7.7) — tests whether the author has thought past "works on my machine, one request at a time" toward the concurrency model a real deployment would need, without expecting that they've already built it (a single-user local tool legitimately doesn't need this yet — the interesting answer is knowing *why* it doesn't need it yet and what would break first if it had to).

8. Interview Quick Reference — One Query Turn, Start to Finish

A user has an active session scoped to a repo (e.g., `torvalds/linux` → `repo_name="linux"`) and types a question into the Streamlit chat box.

1. **Streamlit** (`frontend/streamlit_app.py:106-127`) captures the input via `st.chat_input`, POSTs `{question, session_id, repo_name}` to `POST /chat`.
2. **`routes.chat()`** (`app/api/routes.py:136-170`) resolves `repo_name` from the session if not explicitly passed, calls `save_message(session_id, role="user", ...)` to persist the turn in Postgres immediately, then calls `query_repo(question, session_id, repo_name, strict_mode)`.
3. **`query_repo()`** (`app/graph/rag_graph.py:447-487`) first awaits `get_history(session_id)` from Postgres and formats it into a plain-text block (`format_history_for_prompt`), then builds the initial `RAGState` and calls `graph.invoke(...)` on the compiled `LangGraph`.
4. **`node_intent`** runs `classify_intent()` (rule-based keyword voting by default) to pick one of 5 intents, and in parallel runs `check_need_retrieval()` — a keyword override first, an LLM yes/no gate second — to decide if the codebase needs to be searched at all.
5. If retrieval is needed, **`node_retrieve`** builds a `{"repo": "linux"}` Chroma filter, loads (or reuses the cached) per-repo `BM25Retriever` from `repos/bm25_linux.pkl`, and calls the intent-specific retrieval strategy in `retrieval_agent.py` — for most intents this is `hybrid_search()`: parallel Chroma vector search (Gemini query embedding, cosine similarity) and BM25 keyword search, fused by weighted Reciprocal Rank Fusion (`RRF_K=60`) into one ranked, deduplicated list of ~30 chunks.
6. **`node_rerank`** runs those 30 chunks and the question through a `BAAI/bge-reranker-base` cross-encoder, keeping the top 5 by joint relevance score.
7. In **strict mode**, **`node_grade_chunks`** asks the LLM to label each of those 5 chunks `relevant/ambiguous/irrelevant`; **`node_correct`** drops the `irrelevant` ones and, if *none* survive, rewrites the query and loops back to step 5 once. In fast mode, both nodes are no-ops and every chunk passes through unchanged.
8. **`node_analyse`** groups the surviving chunks by file, regex-scans them for `import/require` statements to detect cross-file relationships, ranks files by chunk count, and writes a short natural-language "here's what's relevant and how it connects" summary.
9. **`node_answer`** builds one large prompt — system rules, chat history, the analysis summary, every chunk as a fenced code block labeled with its file/line range, the question, and an intent-specific instruction ("point to the exact file and line," "walk through step by step," etc.) — and calls Gemini (`gemini-2.5-flash` in this repo's config) to generate the answer.
10. In **strict mode**, **`node_self_critique`** asks the LLM two more yes/no questions — is the answer grounded in the shown code, and does it actually address the question — and if either fails, loops back to step 9 once with a stricter "context-only, no extrapolation" system prompt appended.
11. **`query_repo()`** returns `final_state["final_answer"]` — `{answer, sources, chunks_used, intent, primary_files}` — up through `routes.chat()`, which calls `save_message()` again to persist the assistant's turn (with cited sources and detected intent), then returns the JSON payload to Streamlit, which renders the answer and an expandable "Source files" list.

The one thing to volunteer proactively if asked to trace this live: as written today, if step 4 concludes retrieval *isn't* needed, the flow instead goes straight to `node_direct_answer`, which currently crashes on the `.strip()` bug (§7.1) rather than returning an answer — so the trace above is the "retrieval needed" happy path, and it's worth being upfront that the no-retrieval path currently has a live bug rather than letting an interviewer discover it by asking a follow-up.